

Research Progress Update: Profiling AI Runtime Categories for Shared-RAN Inference Runtime Selection

Category-level profiling and runtime-system selection for edge AI inference on shared RAN infrastructure

Slide 1. What I Have Been Working On

- I am profiling **AI inference runtime categories** to understand which ones are technically suitable for shared RAN-edge execution.
- The runtime categories I am studying are:
 - Transformer / autoregressive (**Llama 3.2 3B + llama.cpp**),
 - Diffusion (**SD1.5 + LCM + TensorRT**),
 - Computer vision (**RTMPose-m + TensorRT**),
 - Voice-related (**Emformer-style streaming ASR + ONNX Runtime Mobile**),
 - Translation (**M2M100-418M + CTranslate2**),
 - Volume rendering (**Instant-NGP + tiny-cuda-nn**).
- For each category, the profiling focus is not just the model architecture, but the **runtime engine and serving stack** used to execute the task.
- Each category is profiled against a runtime suitability scorecard.

Citations: Polese et al. (2025); Feng et al. (2026); profile_doc.md packet summaries.

Slide 2. Suitability Scorecard

- **Preemption resilience**

- Meaning: how safely execution can be interrupted and later resumed.
- Impact: determines how much work is lost when resources are reclaimed.

- **Micro-segmentation**

- Meaning: the smallest execution unit the runtime can expose, such as a token step, operator group, tile, ray batch, or denoising step.
- Impact: determines whether work can fit short execution windows.

- **State parking**

- Meaning: what runtime state must be saved when compute disappears, and how easily that state can be restored.
- Impact: determines pause/resume cost and whether progress survives execution gaps.

- **Tight VRAM compliance**

- Meaning: whether the runtime can stay stable under a strict GPU-memory cap.
- Impact: determines OOM risk, admission stability, and multi-tenant feasibility.

- **Throughput**

- Meaning: how much useful work the runtime can sustain under a given GPU share or lease envelope.
- Impact: determines whether the runtime remains practical when GPU availability is reduced.

Slide 3. How Do I Want to Profile?

- I want to profile the **runtime system**, not only the model family.
- The first profiling strategy is:
 - run the runtime under different GPU-availability configurations inspired by **Weaver-style profiling baselines**,
 - start with **full GPU availability**,
 - then test **reduced or shared GPU availability** such as **MPS-style sharing**,
 - and profile the runtime behavior against the scorecard under each configuration.
- For each configuration, I want to observe:
 - preemption resilience,
 - micro-segmentation,
 - state parking,
 - tight VRAM compliance,
 - throughput.
- The purpose is to see which runtime-system stacks remain viable as GPU availability changes, instead of only measuring them at full-device conditions.

Citations: Weaver workload profiling note; USHER (2024); Orion (2024); vLLM/PagedAttention (2023).

Slide 4. Transformer / Autoregressive

- Category type: streaming, stateful generation with a growing **KV cache**.
- Representative packet-1 anchor: **Llama 3.2 3B + llama.cpp**.
- Current default safe boundary: **token-step**.
- Main profiling question: can the runtime support useful progress, bounded memory growth, and pause/resume around decoder-side state?
- Imported mechanism evidence:
 - **vLLM / PagedAttention**,
 - **Orca**,
 - **DistServe**,
 - **SpotServe**,
 - **CacheGen**.

Citations: PagedAttention / vLLM (2023); Orca (2022); DistServe (2024); SpotServe (2024); CacheGen (2024).

Slide 5. Diffusion

- Category type: one-shot **latent diffusion** with iterative denoising steps.
- Representative packet-1 anchor: **SD1.5 + LCM / LCM-LoRA + TensorRT**.
- Current default safe boundary: **per-request**.
- Strongest imported candidate boundary: **denoising step**.
- Main profiling question: can the runtime make denoising cheap enough, memory-stable enough, and state-aware enough for edge execution under bounded GPU availability?
- Current deployment focus:
 - **TensorRT**,
 - **Triton**,
 - optional **Jetson Platform Services / MIG** as deployment or isolation scaffolds.

Citations: Latent Diffusion Models (2022); LCM (2023); Triton Stable Diffusion tutorial; TensorRT docs.

Slide 6. Computer Vision

- Category type: one-shot, activation-heavy vision inference.
- Representative packet-1 anchor: **RTMPose-m + TensorRT**.
- This category currently covers three completed packet families:
 - **Image segmentation**: workload-aligned anchor **SegFormer / DeepLabV3+**, deployment anchor **YOLO segmentation**.
 - **Pose estimation**: workload-aligned anchor **RTMPose with ViTPose reference**, deployment anchor **RTMPose-m / RTMPose-s**.
 - **Super resolution**: workload-aligned anchor **Real-ESRGAN / SwinIR / HAT**, deployment anchor **Real-ESRGAN**.
- Common runtime pattern: **TensorRT-first**.
- Common profiling question: can the runtime validate any useful **coarse layer-group / tile boundary**, or does it fall back to **per-inference**?

Citations: SegFormer (2021); RTMPose (2023); Real-ESRGAN (2021); USHER (2024); Orion (2024); PPipe (2025).

Slide 7. Voice-related

- This category is **split first** before profiling:
 - streaming ASR,
 - offline ASR,
 - speech translation,
 - TTS.
- Packet-1 branch selected: **streaming ASR**.
- Representative packet-1 anchor: **Emformer-style streaming ASR + ONNX Runtime Mobile**.
- Current default safe boundary: **endpoint / segment**.
- Strongest imported candidate boundary: **audio chunk**.
- Main profiling question: can the runtime carry streaming state efficiently enough on embedded hardware without overstating chunk-level reclaim/resume semantics?

Citations: Emformer (2020); sherpa-onnx streaming docs; ONNX Runtime Mobile docs.

Slide 8. Translation

- Category type: **encoder-decoder seq2seq** with extra fixed **encoder-output state** in addition to decoder KV.
- Representative packet-1 anchor: **M2M100-418M + CTranslate2**.
- Workload-aligned scientific reference: **NLLB-200 distilled-600M**.
- Current default safe boundary: **per-request**.
- Strongest imported candidate boundary: **decoder-step**.
- Main profiling question: can the runtime manage both iterative decoding and the extra persistent encoder-side tensors under the same lease and memory budget?

Citations: M2M-100 (2020); NLLB (2022); CTranslate2 docs; vLLM / Orca / DistServe as imported mechanisms.

Slide 9. Volume Rendering

- Category type: scene-state-aware rendering around **ray-batch** work units.
- Representative packet-1 anchor: **Instant-NGP** + **tiny-cuda-nn**.
- Current best candidate boundary: **ray-batch**.
- Main profiling question: can the system expose useful execution units and compact scene state, even though most primaries are accelerators rather than full serving runtimes?
- Current deployment focus:
 - **native CUDA** / **tiny-cuda-nn**,
 - candidate wrappers like **Triton** + **K3s/Kubernetes**,
 - challengers such as **TensoRF** and **DirectVoxGO**.

Citations: Instant-NGP (2022); NerfAcc (2023); TensoRF (2022); DirectVoxGO (2022).

Slide 10. Current Selection Goal

- For each **category**, I am identifying the **most suitable runtime or runtime-system stack**, not just the most interesting model.
- The runtime will be preferred if it already satisfies the profiling scorecard, or if it can be modified with bounded effort to satisfy it.
- The output of this work is a **category-to-runtime mapping** that supports later prototype design and integration.
- In short: the aim is to select, per category, the stack that is most realistic for safe and efficient inference on shared RAN-edge infrastructure.

Citations: `profile_doc.md` and completed packet set under progress/.

Selected References I

- ① Michele Polese, Niloofar Mohamadi, Salvatore D'Oro, Leonardo Bonati, and Tommaso Melodia. *Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G*. arXiv preprint arXiv:2507.06911, 2025.
- ② Chenyuan Feng et al. *AI-RAN: The Pathway to Future Wireless Networks*. Journal of Information and Intelligence, 2026.
- ③ Sunghyun Jin et al. *End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks*. ACM CoNEXT, 2025.
- ④ Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023.
- ⑤ Gyeong-In Yu et al. *Orca: A Distributed Serving System for Transformer-Based Generative Models*. OSDI, 2022.
- ⑥ Yinmin Zhong et al. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI, 2024.
- ⑦ Gu et al. *SpotServe: Serving Generative Large Language Models on Preemptible Instances*. ASPLOS, 2024.
- ⑧ Rombach et al. *High-Resolution Image Synthesis With Latent Diffusion Models*. CVPR, 2022.
- ⑨ Luo et al. *Latent Consistency Models: Synthesizing High-Resolution Images with Few-Step Inference*. 2023.
- ⑩ Xue et al. *RTMPose: Real-Time Multi-Person Pose Estimation*. 2023.
- ⑪ Liang et al. *SwinIR: Image Restoration Using Swin Transformer*. ICCVW, 2021.
- ⑫ Wang et al. *Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data*. ICCVW, 2021.
- ⑬ Fan et al. *Emformer: Efficient Memory Transformer Based Acoustic Model for Low Latency Streaming Speech Recognition*. 2020.
- ⑭ Fan et al. *No Language Left Behind (NLLB)*. 2022.
- ⑮ Fan et al. *Beyond English-Centric Multilingual Machine Translation*. 2022.
- ⑯ Thomas Müller et al. *Instant Neural Graphics Primitives with a Multiresolution Hash Encoding*. SIGGRAPH, 2022.